

1. Descreva as modificações feitas na organização do BIP I para o BIP II

R: Foi incluso registrador de estado com o resultado das últimas operações aritméticas, eles incluem duas classes adicionais de instrução, uma incondicional que é o jump e outras condicionais. Também foram adicionadas as instruções condicionais.

=====

2. O que significa um rótulo no código assembly?

R: Ao invés de passar diretamente o endereço de memória, passamos um rótulo que referencia aquela posição de memória, sendo mais fácil para montar o código

=====

3. Escreva o código em assembly do BIP II do código C abaixo:

```
``C
if (i==3){
    H = H + 2; // BLOCO 1
} else {
    H = H + 1; // BLOCO 2
}
J = H + 1;
// Fim do programa, pare o processador.
``
```

Código em assembly:

```
``Assembly
LD i
SUBI 3
BEQ BLOCO1 # CASO SEJA 0 VAI PARA O BLOCO 1
             # CASO CONTRÁRIO SEGUE PARA A CONDIÇÃO CONTRÁRIA
LD H
ADDI 1
ST H
JMP FIM_IF

BLOCO1:
LD H
ADDI 2
ST H

FIM_IF: # DEPOIS DO IF VOU PARA O FINAL DA EXECUÇÃO
LD H
ADDI 1
ST J

HLT
``
```

=====

4. Responda as questões considerando o programa abaixo para o BIP II e as seguintes características:

- Os endereços das variáveis X, Y e W são 0x3C, 0x3D e 0x3E, respectivamente.
- Os valores iniciais das variáveis X, Y e W são 5, 5 e 0, respectivamente.

a) O código correspondente em C é:

```C

```
if(x == y) {
 w = x + 2
} else {
 w = 1+x
}
```

...

B)

```
1 LD X
2 SUB Y
3 BEQ EQUAL
4 LDI 1
5 JMP FIM
```

```
6 EQUAL:
7 LDI 2
```

```
8 FIM:
9 ADD X
10 STO W
11 HLT
```

```
8 4 2 1
0 0 1 1 = 3
1 1 0 0 = C
1 1 0 1 = D
1 1 1 0 = E
```

X = 5: 0x3C => 0011 1100

Y = 5: 0x3D => 0011 1101

W = 0: 0x3E => 0011 1110

Portanto:

Endereço | Conteúdo(BIN | HEX)

00 0001 0000 0011 1100 => 103C  
01 0011 0000 0011 1101 => 303D  
02 0100 0000 0000 0101 => 4005 # VAI PARA O ENDEREÇO DE EQUAL  
03 0001 1000 0000 0001 => 1801  
04 0111 0000 0000 0110 => 7006 # VAI PARA O ENDEREÇO DE FIM  
05 0001 1000 0000 0010 => 1802  
06 0010 0000 0011 1100 => 203C  
07 0000 1000 0011 1110 => 083E  
08 0000 0000 0000 0000 => 0000 # FIM DO PROGRAMA

=====

5. Mesma coisa que a questão anterior, mas a c muda que é a tabela:

| instrução | LD    | SUB   | BEQ   | LDI   | ADD   | STO   | HLT   |
|-----------|-------|-------|-------|-------|-------|-------|-------|
| opcode    | 00010 | 00110 | 01000 | 00011 | 00100 | 00001 | 00000 |
| PC        | 1     | 2     | 5     | 6     | 7     | 8     | 8     |
| ACC       | 5     | 0     | 0     | 2     | 7     | 7     | 7     |
| N         | 0     | 0     | 0     | 0     | 0     | 0     | 0     |
| Z         | 0     | 1     | 1     | 1     | 0     | 0     | 0     |
| WrPC      | 1     | 1     | 1     | 1     | 1     | 1     | 0     |
| SELA1     | 0     | 1     | x     | 0     | 1     | x     | x     |
| SELA0     | 0     | 0     | x     | 1     | 0     | x     | x     |
| SELB      | x     | 0     | x     | x     | 0     | x     | x     |
| WrACC     | 1     | 1     | 0     | 1     | 1     | 0     | 0     |
| OpSel     | x     | 1     | x     | x     | 0     | x     | x     |
| RdRam     | 1     | 1     | 0     | 0     | 1     | 0     | 0     |
| WrRam     | 0     | 0     | 0     | 0     | 0     | 1     | 0     |
| WrNz      | 0     | 1     | 0     | 0     | 1     | 0     | 0     |
| Branch    | 0     | 0     | 1     | 0     | 0     | 0     | 0     |

**Opcode:** É o campo de 5 bits da palavra de instrução que identifica qual operação o processador deve executar (ex: 00010 para LD, 00110 para SUB).

**PC (Program Counter):** Registrador de propósito específico que indica o endereço na memória da próxima instrução a ser executada.

**ACC (Accumulator):** Registrador de propósito geral que serve para o armazenamento temporário de dados e atua como operando implícito nas instruções do processador.

**N (Negative):** Bit de estado (STATUS.N) adicionado no BIP II que indica se o resultado da última operação aritmética na ULA foi menor que zero (negativo).

**Z (Zero):** Bit de estado (STATUS.Z) adicionado no BIP II que indica se o resultado da última operação aritmética na ULA foi exatamente igual a zero.

**WrPC (Write PC):** Sinal de controle que habilita a atualização do registrador PC. Quando em 1, o PC avança para a próxima instrução; quando em 0 (como na instrução HLT), a execução é paralisada.

**SELA1 e SELA0 (SelA[1..0]):** Sinais de controle de 2 bits para o multiplexador A, que define de onde vem o dado que será gravado no ACC. Ele seleciona entre o dado da Memória, a Extensão de Sinal (Imediato) ou a saída da Unidade Aritmética.

**SELB:** Sinal de controle para o multiplexador na entrada da Unidade Aritmética, utilizado para rotear corretamente os operandos para as operações matemáticas.

**WrACC (Write ACC):** Sinal de controle que, quando ativado, permite (habilita) a gravação de um novo valor dentro do registrador Acumulador.

**OpSel (Op):** Sinal que define qual operação matemática a Unidade Aritmética deve realizar (ex: 0 para soma em ADD, 1 para subtração em SUB).

**RdRam (Read RAM):** Sinal que habilita a leitura de um valor contido na Memória de Dados para que seja carregado no processador.

**WrRam (Write RAM):** Sinal que habilita a escrita (gravação) do valor atual do ACC em um endereço da Memória de Dados (usado na instrução STO).

**WrNz:** Sinal exclusivo do BIP II que habilita a gravação/atualização dos bits de estado N e Z. É ativado apenas durante instruções que envolvem operações aritméticas na ULA.

**Branch:** Sinal de controle do BIP II que gerencia o desvio do programa. Ele cruza as informações do opcode das instruções de pulo (como BEQ ou JMP) com o status das flags N e Z para decidir se o PC deve ser sobrescrito com um novo endereço de destino.